TUTORIAL

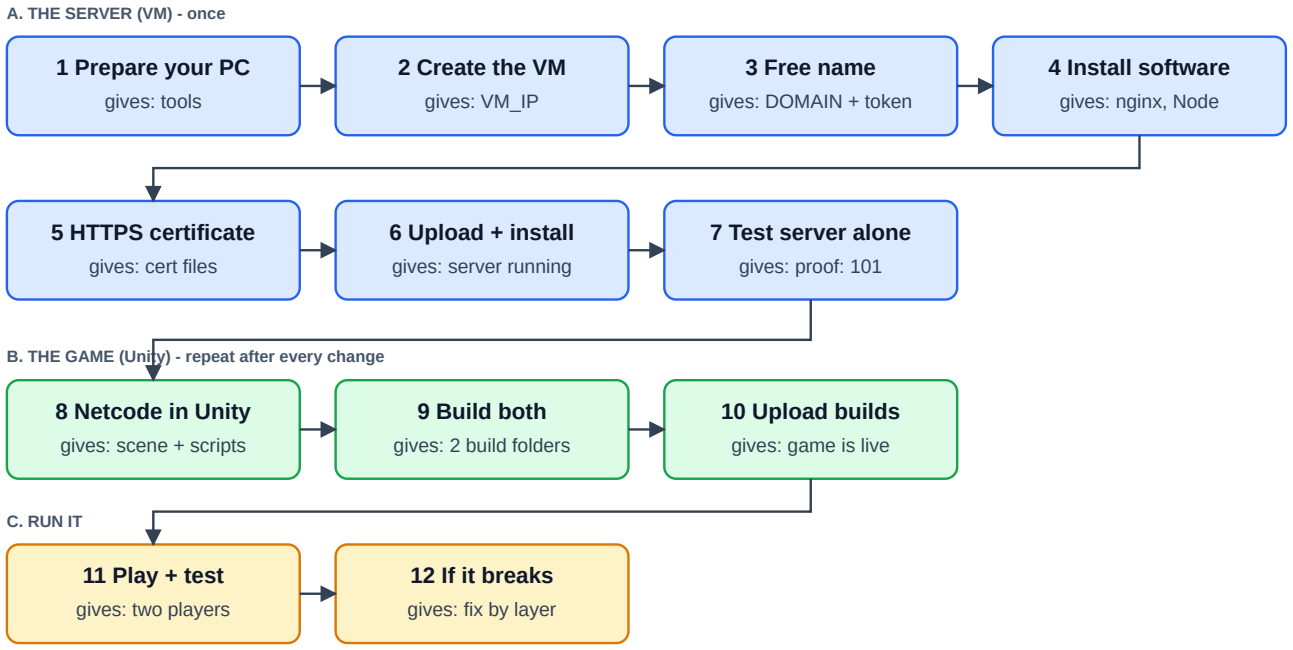
# Free multiplayer Unity WebGL game: step by step

From an empty VirtualBox VM to two browsers playing in the same room

> **How this guide is built.** It is one road with 12 steps. Every step says what it **needs** from earlier steps and what it **gives** to later ones, shows where you are on the road, and ends with a check you can run before moving on.

Repository: github.com/SynokJ/multiplayer_maze | Unity 6 | Netcode for GameObjects 2.x | Ubuntu + nginx + Node 22

# The road in one picture




Every arrow means: the next step needs what the previous step produced.

A Set up the server (once) | B Build the game (repeat after every change) | C Run it and test

**What has been run and what has not.**

- **Run for real (in the research sandbox):** the Node lobby (create room, list rooms, join, clean-up) with a stand-in room. The stand-in answered a WebSocket echo.
- **Written from documentation, never run:** the nginx file, the install scripts, the VirtualBox and certificate commands, and all Unity C# scripts. There was no Unity, VirtualBox or nginx in the sandbox.
- **Still a guess:** that the browser really sends the room path (`/r/7003`) and that nginx passes Unity's data through unchanged.

That is why the steps are ordered so that **errors show up early**: Step 7 tests the server without Unity, Step 11 tests the whole chain. If Step 11 fails on routing, there are two ready fallbacks (see Step 12).

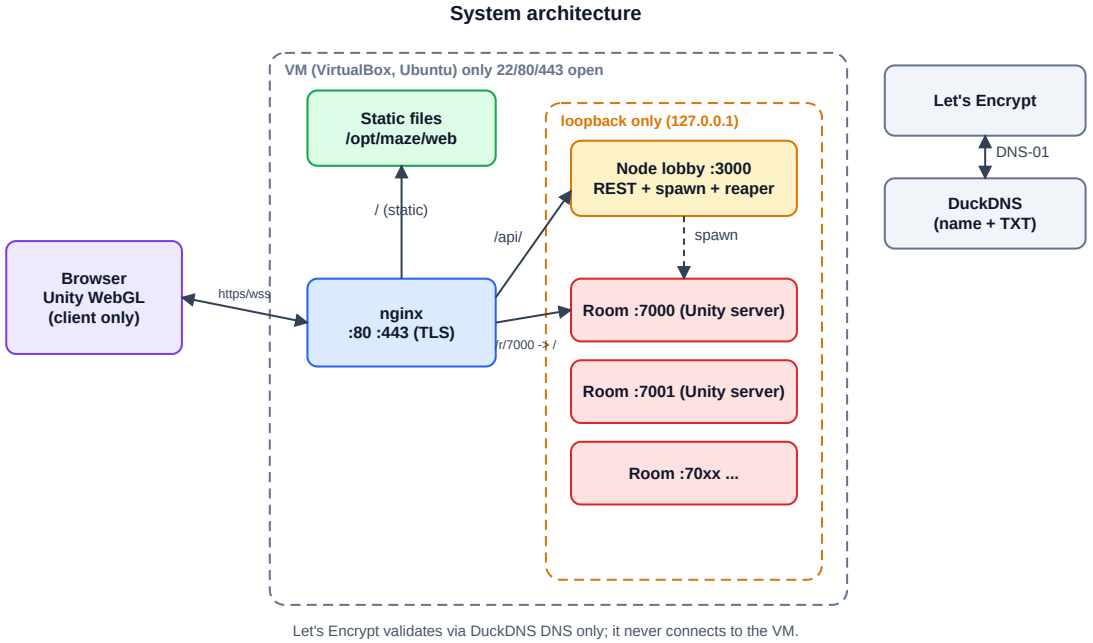
# The big picture in 60 seconds

You are building three things that work together:

| Piece | What it is | Where it runs |
|---|---|---|
| **The game** | Your Unity project, built as a WebGL page. Players open it in a browser. | The player's browser |
| **The door (nginx)** | Serves the game page, answers the secure `https://` and `wss://` traffic, and forwards it inside. | The VM, port 443 |
| **The lobby + rooms** | A small Node program (the lobby) creates rooms. Each room is one copy of your Unity game running without a screen (the Linux server build). | The VM, hidden behind nginx |

**A player's journey, in 5 lines:**

1. The player opens `https://DOMAIN/`. nginx sends the game page.
2. The player types a name and presses **Create room**. The game asks the lobby.
3. The lobby starts a new room program and answers with a room code, a path like `/r/7003`, and a secret join token.
4. The game connects to `wss://DOMAIN/r/7003`. nginx forwards that to the room program.
5. The room checks the token and lets the player in. A friend types the code and joins the same room.




The VM from the outside: only nginx is open. Lobby and rooms hide behind it.

# How the steps are connected

Each step produces something that a later step consumes. If a step fails, look at what it needed.

| Step | Name | What it hands on | Used in step |
|---|---|---|---|
| 1 | **Prepare your PC** | Unity with WebGL + Linux server modules, VirtualBox, ssh | 2, 8, 9 |
| 2 | **Create the VM** | VM_IP and a working ssh login | 3, 4, 6 |
| 3 | **Free name** | DOMAIN (points to VM_IP) and DUCK_TOKEN | 5, 6 |
| 4 | **Install software** | nginx, Node 22, firewall | 5, 6 |
| 5 | **HTTPS certificate** | Certificate files for DOMAIN | 6 |
| 6 | **Upload + install** | Lobby, nginx site and service running (with a stand-in room) | 7, 10 |
| 7 | **Test server alone** | Proof that HTTPS and WebSocket routing work | 10, 11 |
| 8 | **Netcode in Unity** | A scene with network components and scripts | 9 |
| 9 | **Build both** | WebGL folder and Linux server folder | 10 |
| 10 | **Upload builds** | The real game online (stand-in replaced) | 11 |
| 11 | **Play + test** | Two players in one room | 12 |
| 12 | **If it breaks** | A checklist, from the outside in | - |

## Your values sheet

Write these down as you create them. They appear again and again.

| Value | Example | Created in step | Used in steps |
|---|---|---|---|
| `VM_IP` | 192.168.1.50 | **2** | 3, 6, 10 |
| `VM_USER` | vmadmin | **2** | 6, 10 |
| `DOMAIN` | mymaze.duckdns.org | **3** | 5, 6, 7, 10, 11 |
| `DUCK_TOKEN` | a1b2c3... (secret) | **3** | 5 |
| `TOKEN_SECRET, INTERNAL_KEY` | made automatically | **6** | 6 (stays on the VM) |
| `Build version` | 0.1.0 | **9** | 9, 10 |

A THE SERVER

# STEP 1 of 12  Prepare your PC

1 2 3 4 5 6 7 8 9 10 11 12

A server  B game  C run

**Why this step:** Everything else is driven from your PC: Unity builds the game, VirtualBox hosts the server, ssh and scp talk to it.

**NEEDS**

- Nothing. This is the start.

→ THIS STEP →

**GIVES**

- Tools you will use in steps 2, 8 and 9.

## Do

Install, in this order:

1. **Unity Hub**, then **Unity 6000.6.6f1** (the version your project uses). In the install dialog tick the modules **WebGL Build Support** and **Linux Dedicated Server Build Support**. If you also see **Linux Build Support (Mono)**, tick it.
2. **VirtualBox 7.x** (free, from virtualbox.org). In your computer's BIOS, hardware virtualization must be on.
3. **An ssh client.** Windows 10/11 already has `ssh` and `scp`: open **PowerShell** and type `ssh -V`. macOS and Linux have them already.
4. **Git**, and clone this repository so you have the `server/`, `deploy/` and `unity-scripts/` folders:

```
git clone https://github.com/SynokJ/multiplayer_maze.git
cd multiplayer_maze
git checkout claude/laughing-galileo-n6jfav
```

**Your PC must be Intel/AMD (x86-64).** Unity builds the Linux server for x86-64. A Mac with an Apple chip cannot run it in this plan.

**CHECK IT WORKED**

`ssh -V` prints a version, VirtualBox starts, and Unity Hub lists the two modules for 6000.6.6f1.

**IF IT DOES NOT**

A module is missing: in Unity Hub open **Installs**, the gear icon, **Add modules**.

**NEXT → Step 2 creates the server computer that you will log in to.**

A THE SERVER

# STEP 2 of 12  Create the VM (your test server)

1 2 3 4 5 6 7 8 9 10 11 12

A server · B game · C run

**Why this step:** The VM is a small Linux computer inside your PC. It plays the role of the real server: it will hold nginx, the lobby and the rooms.

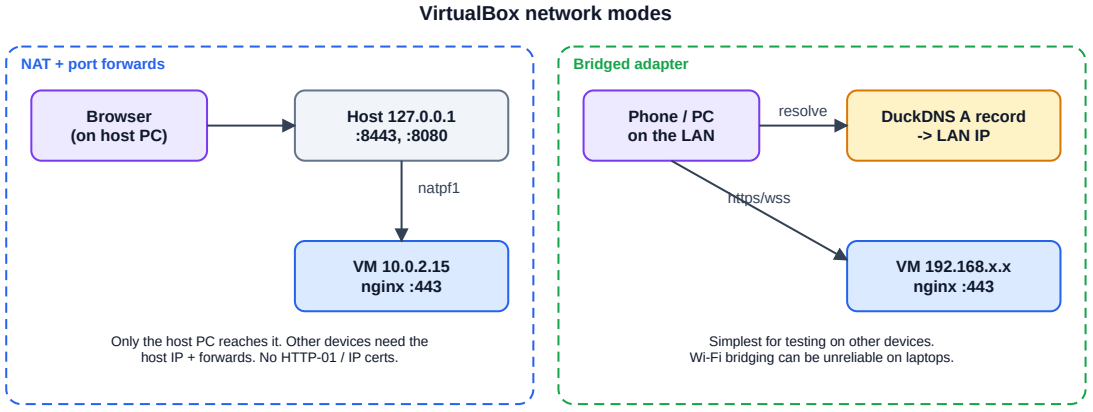
**NEEDS**

- Step 1: VirtualBox and an ssh client.
- The Ubuntu Server ISO (download "Ubuntu Server" LTS from ubuntu.com).

→ THIS STEP →

**GIVES**

- **VM_IP**, the address of the VM (used in steps 3, 6, 10).
- **VM_USER** and a working `ssh` login.




Bridged (recommended) or NAT: how the VM is reached

## Do

**2.1 Create the machine.** In VirtualBox click **New**. Name: `maze-vm`. Pick the ISO. Tick **Skip Unattended Installation**. Memory **4096 MB**, **2 CPUs**, disk **20 GB**.

**2.2 Give it a network address.** Open the VM **Settings, Network, Adapter 1**. Set **Attached to: Bridged Adapter** and choose the network card you use for the internet. The VM then gets its own address on your home network, like a real PC.

**2.3 Install Ubuntu. Start** the VM and follow the installer. Keep the defaults, except:

- Use the username `vmadmin` and a password you remember.
- Tick **Install OpenSSH server**.
- Skip the optional extra software list.

**2.4 First login.** When the installer asks, **reboot**, and log in on the VM window with `vmadmin`. Then find the VM's address:

```
hostname -I
```

The first number, like `192.168.1.50`, is your **VM_IP**. Write it down.

**2.5 Log in from your PC.** Use your own PC's terminal (not the VM window). From now on you work in this window:

```
ssh vmadmin@192.168.1.50      # replace with your VM_IP
```

**Bridged or NAT?** Bridged is easiest, and phones on the same Wi-Fi can reach the VM. If your network card cannot be bridged (some Wi-Fi cards), use **NAT** with port forwards instead: forward host `2222` to guest `22`, and host `8443` to guest `443`. Then log in with `ssh -p 2222 vmadmin@127.0.0.1`, and in Step 6 you must also set `PUBLIC_PORT=8443` (the install script explains where).

**CHECK IT WORKED**

`ssh vmadmin@VM_IP` from your PC shows a Ubuntu welcome text and a prompt like `vmadmin@maze-vm:~$`.

**IF IT DOES NOT**

No address from `hostname -I` on bridged: choose another network card in Settings, or use NAT. Connection refused on ssh: OpenSSH server was not ticked; on the VM run `sudo apt install -y openssh-server`.

**NEXT → Step 3 gives this address a free name, so browsers can reach it by name and trust it.**

A THE SERVER

# STEP 3 of 12 Get a free name

1 2 3 4 5 6 7 8 9 10 11 12

A server B game C run

**Why this step:** A trusted HTTPS certificate is issued for a **name**, not for a bare number. DuckDNS gives you a free name like `mymaze.duckdns.org` and points it to your VM.

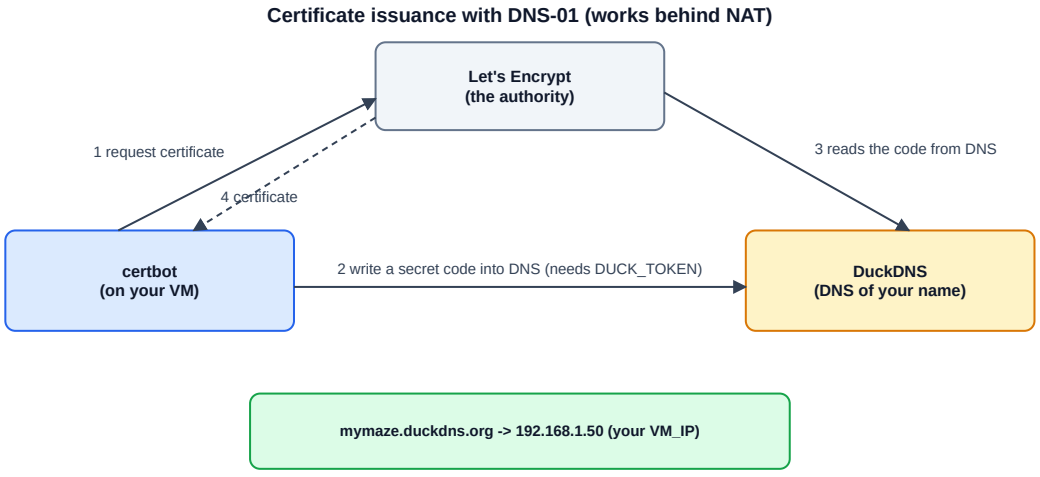
**NEEDS**

- Step 2: **VM_IP**.

→ THIS STEP →

**GIVES**

- **DOMAIN**, e.g. `mymaze.duckdns.org` (used in steps 5, 6, 7, 10, 11).
- **DUCK_TOKEN**, your secret key (used only in Step 5).




How the certificate is proven without opening your VM to the internet

## Do

1. Go to **duckdns.org** and sign in (Google, GitHub, etc. - it is free).
2. In the box **sub domain** type a name, for example `mymaze`, and click **add domain**. Your **DOMAIN** is now `mymaze.duckdns.org`.
3. In the row of that domain, find the field **current ip**. Type your **VM_IP** (for example `192.168.1.50`) and click **update ip**.
4. Copy the **token** shown at the top of the DuckDNS page. This is your **DUCK_TOKEN**. Keep it secret and never put it in git.

**Is it OK that the address is private (192.168...)?** Yes. The certificate in Step 5 is proven through a DNS record, so Let's Encrypt never needs to reach your VM from the internet. That is why this works behind your router.

**CHECK IT WORKED**

On your PC run `nslookup mymaze.duckdns.org` (use your DOMAIN). It must print your VM_IP.

**IF IT DOES NOT**

It prints nothing or a different address: wait one minute and retry, or try `nslookup mymaze.duckdns.org 8.8.8.8`. Some routers block private answers ("DNS rebind protection"); the second command tells you if that is the cause.

**NEXT → Step 4 installs the programs on the VM. Step 5 then uses DOMAIN and DUCK_TOKEN to get the certificate.**

A THE SERVER

# STEP 4 of 12 Install the software on the VM




**Why this step:** The VM is empty. It needs the door (nginx), the runtime for the lobby (Node 22) and a firewall that lets in only what you need.

**NEEDS**

- Step 2: your `ssh` login to the VM.

→ THIS STEP →

**GIVES**

- nginx, Node 22, a firewall, and a clock that is correct (needed by certificates).

## Do

Do this **inside the ssh window** (Step 2):

```
sudo apt update && sudo apt -y upgrade
sudo apt install -y nginx python3-venv curl ufw openssl
# Node 22 (from NodeSource)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
# firewall: only ssh, http and https. Allow ssh FIRST or you lock yourself out.
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
```

Ports 3000 (lobby) and 7000-7099 (rooms) stay closed on purpose. Only nginx on the VM talks to them.

**CHECK IT WORKED**

`node -v` prints `v22...`, `nginx -v` prints a version, `sudo ufw status` lists 22, 80 and 443, and `timedatectl` says `System clock synchronized: yes`.

**IF IT DOES NOT**

The NodeSource line fails: check the VM has internet (`ping -c1 deb.nodesource.com`). The clock is wrong: `sudo timedatectl set-ntp true`.

**NEXT → Step 5 asks Let's Encrypt for the certificate. nginx will use it in Step 6.**

A THE SERVER

# STEP 5 of 12 Get the HTTPS certificate




**Why this step:** Browsers refuse `ws://` on an `https://` page, and they only trust certificates from a known authority. Let's Encrypt gives you one for free.

**NEEDS**

- Step 3: **DOMAIN** and **DUCK_TOKEN**.
- Step 4: Python venv and a working clock.

→ THIS STEP →

**GIVES**

- Certificate files in `/etc/letsencrypt/live/DOMAIN/` (the nginx file in Step 6 points to them).
- Automatic renewal.

## Do

In the ssh window. Edit the first three lines, then paste the whole block:

```
DOMAIN=mymaze.duckdns.org          # your DOMAIN from Step 3
DUCK_TOKEN=paste-your-token-here   # from Step 3
EMAIL=you@example.com              # for expiry warnings

sudo python3 -m venv /opt/certbot
sudo /opt/certbot/bin/pip install certbot certbot-dns-duckdns
echo "dns_duckdns_token=$DUCK_TOKEN" | sudo tee /root/duckdns.ini >/dev/null
sudo chmod 600 /root/duckdns.ini

# 1) rehearsal against the test server (no limits, certificate is not trusted)
sudo /opt/certbot/bin/certbot certonly --non-interactive --agree-tos --email $EMAIL \
  --preferred-challenges dns --authenticator dns-duckdns \
  --dns-duckdns-credentials /root/duckdns.ini --dns-duckdns-propagation-seconds 60 \
  -d $DOMAIN --staging

# 2) the real certificate (same command, without --staging, plus --force-renewal)
sudo /opt/certbot/bin/certbot certonly --non-interactive --agree-tos --email $EMAIL \
  --preferred-challenges dns --authenticator dns-duckdns \
  --dns-duckdns-credentials /root/duckdns.ini --dns-duckdns-propagation-seconds 60 \
  -d $DOMAIN --force-renewal

# 3) renew automatically and tell nginx
sudo mkdir -p /etc/letsencrypt/renewal-hooks/deploy
printf '#!/bin/sh\nsystemctl reload nginx\n' | sudo tee /etc/letsencrypt/renewal-
hooks/deploy/reload-nginx.sh >/dev/null
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
echo '0 3 * * * root /opt/certbot/bin/certbot renew -q' | sudo tee /etc/cron.d/certbot-renew
>/dev/null
```

Why the rehearsal first: Let's Encrypt limits failed attempts. The `--staging` run proves your token and DNS work before you spend a real attempt.

**CHECK IT WORKED**

```
sudo ls /etc/letsencrypt/live/$DOMAIN/
sudo openssl x509 -in /etc/letsencrypt/live/$DOMAIN/fullchain.pem -noout -issuer -dates
```

You see `fullchain.pem` and `privkey.pem`, and the issuer says **Let's Encrypt** (not "STAGING").

**IF IT DOES NOT**

"Timed out waiting for DNS": raise `--dns-duckdns-propagation-seconds` to 120. After the rehearsal the real run complains about the existing certificate: run `sudo /opt/certbot/bin/certbot delete --cert-name $DOMAIN` and repeat part 2.

**NEXT → Step 6 installs the server programs. The install script refuses to continue unless this certificate exists.**

A THE SERVER

# STEP 6 of 12 Upload and install the server files




**Why this step:** This is the "upload" step. It moves the lobby, the nginx settings and the service definition from your repository onto the VM and switches everything on.

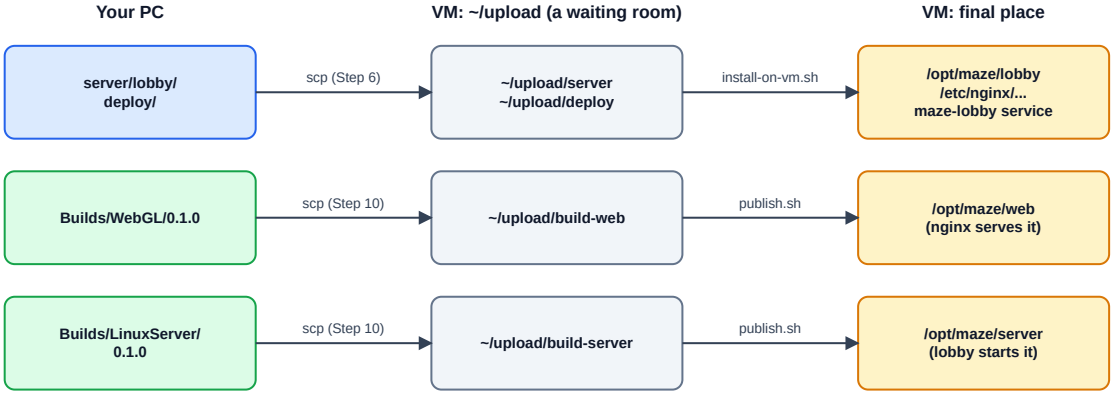
**NEEDS**

- Step 2: **VM_IP** and **VM_USER**.
- Step 3: **DOMAIN**.
- Step 4: nginx and Node installed.
- Step 5: the certificate.
- Step 1: the cloned repository on your PC.

→ THIS STEP →

**GIVES**

- The lobby running as a service, nginx serving DOMAIN, secrets created, and a **stand-in room** so you can test before Unity exists.




scp copies files from your PC. The scripts on the VM move them into place and restart the services.

Where your files travel (Steps 6 and 10)

## Do

**On your PC**, in the repository folder, upload two folders:

```
VM=vmadmin@192.168.1.50            # VM_USER@VM_IP
ssh $VM "mkdir -p ~/upload"
scp -r server deploy $VM:~/upload/
```

**On the VM** (ssh window), fix possible Windows line endings, then run the installer with your DOMAIN:

```
find ~/upload -type f \( -name '*.sh' -o -name '*.service' -o -name '*.template' \) -exec sed -i
's/\r$//' {} +
sudo bash ~/upload/deploy/install-on-vm.sh mymaze.duckdns.org
```

What the installer does, in plain words:

| Action | Result |
|---|---|
| Creates a service user `maze` and folders in `/opt/maze` | Rooms and the lobby do not run as an administrator |
| Copies the lobby and runs `npm install` | Lobby code at `/opt/maze/lobby` |
| Writes `/etc/maze/lobby.env` with two random secrets | Secrets stay only on the VM |
| Fills your DOMAIN into the nginx template, tests it, reloads nginx | Door open on port 443 |
| Installs and starts the `maze-lobby` service | Lobby survives reboots |

The lobby is told to start `fake-room.sh` as its room program. It is a tiny stand-in for the Unity server. Step 10 switches it to the real one automatically.

**If you use NAT** (Step 2): open `/etc/maze/lobby.env` and set `PUBLIC_PORT=8443`, then `sudo systemctl restart maze-lobby`.

**CHECK IT WORKED**

```
sudo systemctl status maze-lobby --no-pager | head -5
curl -s http://127.0.0.1:3000/api/health
curl -s https://mymaze.duckdns.org/api/health
```

Both `curl` lines print `{"ok":true,"rooms":0}`. The second one proves HTTPS, the certificate and nginx forwarding in one shot.

**IF IT DOES NOT**

"No certificate": go back to Step 5. `nginx -t` fails: the error names the line; the usual cause is a typo in DOMAIN. Service not active: `sudo journalctl -u maze-lobby -n 30 --no-pager`.

**NEXT → Step 7 proves the server half works, with no Unity involved.**

A THE SERVER

# STEP 7 of 12 Test the server by itself

1 2 3 4 5 6 **7** 8 9 10 11 12

A server · B game · C run

**Why this step:** Find server problems now, while there is no game to blame. You will create a room, then open a WebSocket to it **through nginx**, exactly as the browser will later.

**NEEDS**

- Step 6: lobby, nginx and the stand-in room program running.

→ THIS STEP →

**GIVES**

- Proof that HTTPS, the lobby, the path rewrite `/r/7000` and WebSocket upgrade all work.

## Do

All commands run on the VM (ssh window). Use your own DOMAIN.

```
DOMAIN=mymaze.duckdns.org

# 1) ask the lobby for a room
curl -s -X POST https://$DOMAIN/api/rooms -H "content-type: application/json" -d
'{"playerName":"Ann"}'
```

The answer contains a line like `"path":"/r/7000"`. The number is the room's port. Use it below.

```
# 2) open a WebSocket to that room through nginx (stops after 3 seconds)
curl -i -N --max-time 3 --http1.1 \
  -H "Connection: Upgrade" -H "Upgrade: websocket" \
  -H "Sec-WebSocket-Version: 13" -H "Sec-WebSocket-Key: SGVsbG8sIHdvcmxkIQ==" \
  https://$DOMAIN/r/7000

# 3) see what nginx did
sudo tail -n 3 /var/log/nginx/rooms.log
```

**CHECK IT WORKED**

Step 2 prints `HTTP/1.1 101 Switching Protocols`. Step 3 shows a line with `"GET /r/7000"`, `101` and `upstream=127.0.0.1:7000`.

**Why 101 proves the rewrite:** the stand-in room only accepts the path `/`. Your request used `/r/7000`. It got in, so nginx turned `/r/7000` into `/`.

**IF IT DOES NOT**

`502`: the room program is not running (look at `sudo journalctl -u maze-lobby -n 30`). `404`: the path number is outside 7000-7099. `400` / `426`: copy the curl command exactly, including the headers.

**NEXT → The server half is done. Steps 8-10 build the game that will use it.**

B THE GAME

# STEP 8 of 12 Add networking to the Unity project




**Why this step:** Unity needs the networking package and a few scripts. The scripts connect the browser game to the lobby (Step 6) and make the room program check the join token.

**NEEDS**

- Step 1: Unity 6000.6.6f1 and the cloned repository.
- Step 6: the lobby answers on `/api/rooms` (the script `LobbyClient` calls it).

→ THIS STEP →

**GIVES**

- A scene with `NetworkManager`, a player prefab and the scripts, ready to build in Step 9.

## Do

Open the folder `multiplayer game` with Unity Hub, then:

**1. Install the package.** `Window > Package Manager > + > Install package by name`. Name: `com.unity.netcode.gameobjects`, version `2.13.3`. (This also installs Unity Transport.)

**2. Copy the scripts** from the repository folder `unity-scripts/` into your project:

| Copy from | Copy to (in Unity) | What it does |
|---|---|---|
| `Net/ServerBootstrap.cs` | `Assets/Scripts/Net/` | Runs only in the Linux server. Opens the port, checks the join token, reports to the lobby every 5 seconds. |
| `Net/ClientConnect.cs` | `Assets/Scripts/Net/` | Runs in the browser. Connects with the room path and token. |
| `Net/LobbyClient.cs` | `Assets/Scripts/Net/` | Runs in the browser. Draws the name field and the Create / Join buttons. |
| `Editor/BuildScript.cs` | `Assets/Editor/` | Adds the menu `Build > WebGL` and `Build > Linux Server` (Step 9). |
| `Gameplay/*.cs` | `Assets/Scripts/Gameplay/` | Examples of synced variables and remote calls (see "How it works"). |

**3. Set up the scene.** Open `SampleScene` and:

1. Create an empty GameObject named `Net`. Add the component **NetworkManager**. In its inspector, under the transport choice, select **UnityTransport**.
2. On the **UnityTransport** component tick **Use Web Sockets**. Leave the other fields alone: the scripts set address, port and path while running.
3. Add to the same `Net` object: **ServerBootstrap**, **ClientConnect**, **LobbyClient**.
4. Make a player: `GameObject > 2D Object > Sprites > Square`. Add the components **NetworkObject**, **NetworkTransform**, **PlayerState**, **MoverNet**. Drag it into the Project window to make a prefab, then delete it from the scene.

5. `Right-click in Project > Create > Netcode > Network Prefabs List`. Add your player prefab to that list.
6. Select `Net`. In the **NetworkManager** inspector: put the list under **Network Prefabs Lists**, and the prefab into **Player Prefab**.
7. `File > Build Profiles` (or Build Settings): make sure `SampleScene` is in the scene list.
8. `Edit > Project Settings > Player > Version`: set `0.1.0`.

Menu names can differ slightly between package versions. If a name is not found, search for the component name in the inspector's **Add Component** box.

**CHECK IT WORKED**

The Console shows **no red errors**. The scene has the `Net` object with four components.

**IF IT DOES NOT**

Errors mentioning `Unity.Netcode`: the package from part 1 is not installed yet. "Missing script" on `Net`: the script file name must match the class name exactly. Not sure a UI name exists: the scripts do not depend on the labels, only on the components.

**NEXT → Step 9 turns this scene into the two programs the server needs.**

B THE GAME

# STEP 9 of 12 Build the game twice




**Why this step:** You need two programs from the same code: the **WebGL page** that players open, and the **Linux server** that the lobby starts once per room. They must come from the same version, or the room refuses the player.

**NEEDS**

- Step 8: the scene, the scripts and `Assets/Editor/BuildScript.cs`.
- Step 1: the WebGL and Linux server modules.

→ THIS STEP →

**GIVES**

- Folder `Builds/WebGL/0.1.0/` and folder `Builds/LinuxServer/0.1.0/` (used in Step 10).

## Do

In the Unity menu bar:

1. `Build > WebGL`. Wait until the Console says `Build OK WebGL`.
2. `Build > Linux Server`. Wait until it says `Build OK StandaloneLinux64`.

The first build of each kind takes several minutes. The script sets Gzip compression and hashed file names for you.

You now have, inside `multiplayer game/`:

```
Builds/WebGL/0.1.0/            index.html, Build/ (large .gz files), TemplateData/
Builds/LinuxServer/0.1.0/      MazeServer.x86_64  and  MazeServer_Data/
```

**Every time you change code or the scene, build both again.**

**CHECK IT WORKED**

Both folders exist and the Linux folder contains `MazeServer.x86_64`.

**IF IT DOES NOT**

"Linux Dedicated Server" is not installed: add the module in Unity Hub (Step 1). "Another instance is open": close any second Unity window on the project. Linux server link error mentioning IL2CPP: switch the server to the Mono scripting backend in Project Settings.

**NEXT → Step 10 puts both folders on the VM.**

B THE GAME

# STEP 10 of 12 Upload the builds and go live




**Why this step:** The VM still runs the stand-in room. Now you replace it with your real Unity server and publish the game page.

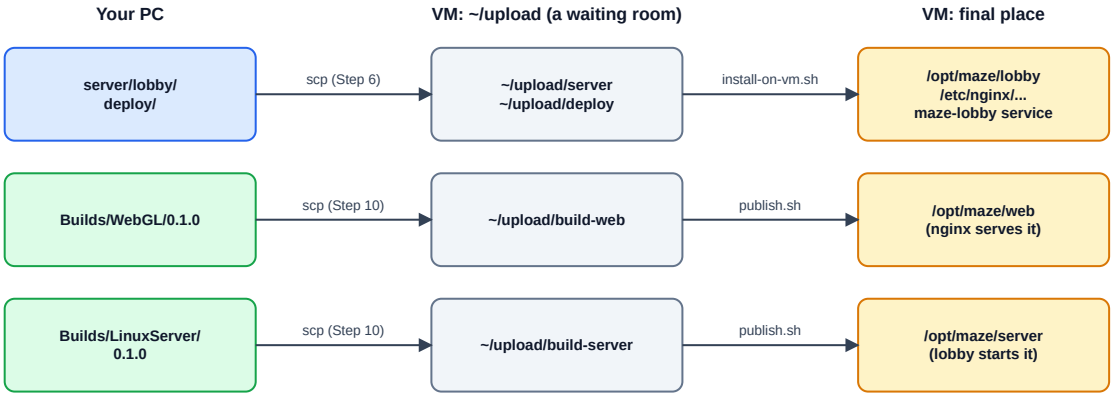
**NEEDS**

- Step 9: both build folders.
- Step 2: **VM_USER**, **VM_IP**.
- Step 6: `/opt/maze/bin/publish.sh`, installed by the installer.

→ THIS STEP →

**GIVES**

- Your game at `https://DOMAIN/`, with the lobby starting real Unity rooms.




scp copies files from your PC. The scripts on the VM move them into place and restart the services.

Where your files travel (Steps 6 and 10)

## Do

**On your PC**, in the repository folder:

```
VM=vmadmin@192.168.1.50            # VM_USER@VM_IP
V=0.1.0                            # the version from Step 9
ssh $VM "rm -rf ~/upload/build-web ~/upload/build-server"
scp -r "multiplayer game/Builds/WebGL/$V"       $VM:~/upload/build-web
scp -r "multiplayer game/Builds/LinuxServer/$V" $VM:~/upload/build-server
```

**On the VM:**

```
sudo /opt/maze/bin/publish.sh
```

The script copies the page to `/opt/maze/web`, the server to `/opt/maze/server`, sets the file permissions, **points the lobby at your real server program**, reloads nginx and restarts the lobby.

Optional quick check that the real server starts (prints `ROOM_READY`, stop with Ctrl+C):

```
cd /opt/maze/server && sudo -u maze ./MazeServer.x86_64 -batchmode -nographics -port 7099 -
roomId TEST1 -logFile -
```

**CHECK IT WORKED**

```
curl -sI https://mymaze.duckdns.org/ | head -1
curl -sI https://mymaze.duckdns.org/Build/$(ls /opt/maze/web/Build | grep wasm.gz | head -1)
| grep -i -E "content-type|content-encoding"
```

First line: `200`. Second: `content-type: application/wasm` and `content-encoding: gzip`.

**IF IT DOES NOT**

`403` on the page: run the publish script again (it sets permissions). "Permission denied" starting the server: the script runs `chmod +x` for you, so check the file name is exactly `MazeServer.x86_64`. Wrong `content-encoding`: the nginx template must be the one from the repository; see Step 12.

**NEXT → Step 11: open the game in two windows and play.**

C RUN IT

# STEP 11 of 12 Play and test with two players




**Why this step:** This is the real test: the whole chain from browser to nginx to lobby to a real Unity room, and a second player joining it.

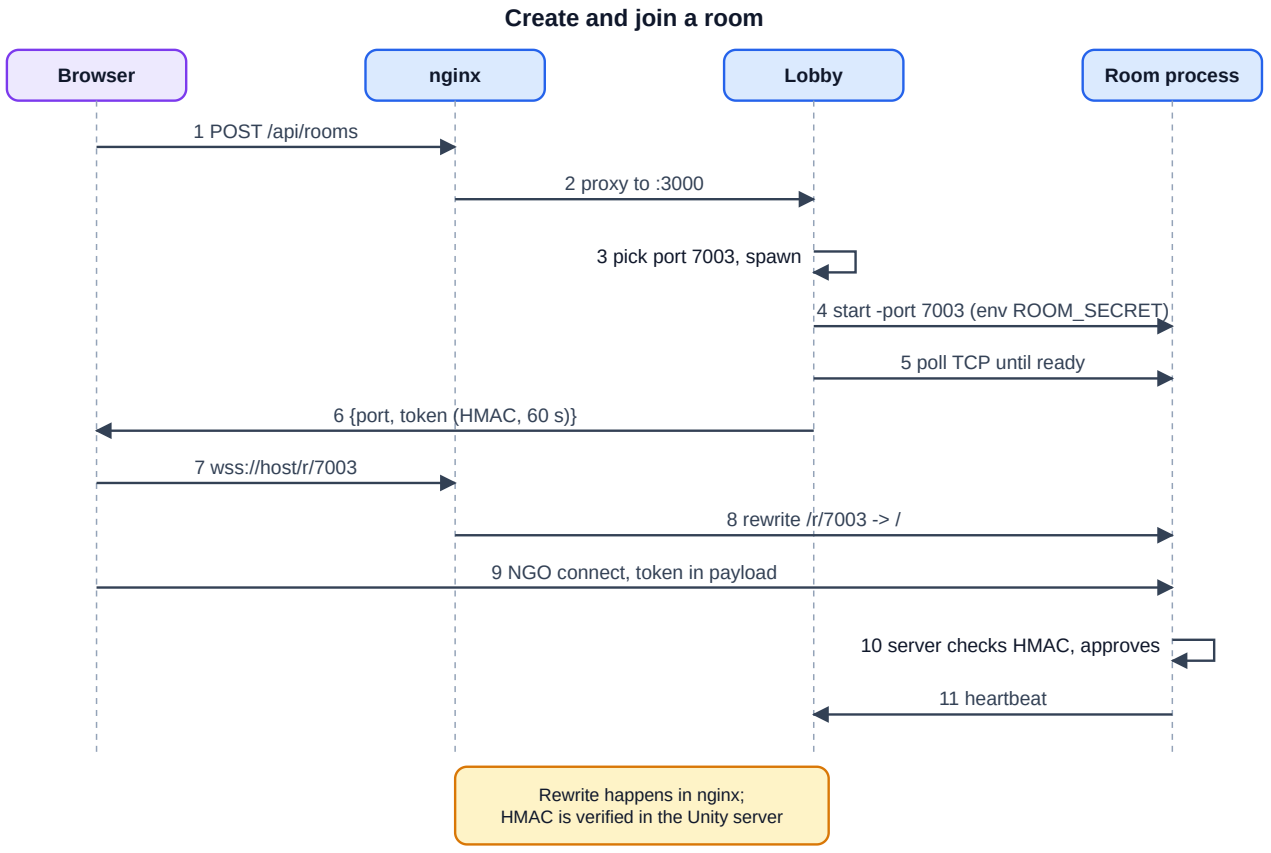
**NEEDS**

- Step 10: game published.
- Step 7: you already know the server half works, so a failure here points to the Unity side.

→ THIS STEP →

**GIVES**

- Confidence: two browsers share one room. Or a clear symptom to take to Step 12.




What happens when a player presses Create room

## Do

**11.1 Watch the server.** On the VM, open two ssh windows and run one command in each:

```
sudo journalctl -u maze-lobby -f              # lobby and room messages
sudo tail -f /var/log/nginx/rooms.log         # every connection to a room
```

**11.2 Open the game twice.** On your PC open `https://mymaze.duckdns.org/` (your DOMAIN) in a normal window, and again in a **private window**, side by side. Hidden tabs slow down in browsers, so keep both visible.

**11.3 Player 1.** Type a name, press **Create room**. The text changes to `Connected to room XXXXX`. Note the five-letter code.

**11.4 Player 2.** Type another name, enter the code, press **Join room**. Both windows show `Players in room: 2`.

**11.5 Move.** Click inside a window and press **W A S D**: the square moves on both screens.

What you should see in the logs:

| Where | Line | Means |
|---|---|---|
| lobby log | `ROOM_READY ABCDE 7000` | The Unity room started and listens |
| rooms.log | `"GET /r/7000" 101 upstream=127.0.0.1:7000` | nginx upgraded the browser's connection and forwarded it |
| lobby log | no "bad token" lines | The token check passed |

How to read a problem:

| What you see | Meaning | Go to |
|---|---|---|
| `rooms.log` shows `GET /r/7000 101` but the game says "disconnected" | Path and TLS are fine; the room rejected the player or dropped data | Step 12, rows 5-6 |
| `rooms.log` shows `GET /` instead of `/r/7000` | The browser did not send the path | Step 12, "If routing fails" |
| `rooms.log` shows nothing | The browser never reached nginx, or mixed content blocked it | Step 12, rows 1-3 |

**CHECK IT WORKED**

Two windows, one room code, `Players in room: 2`, the square moves in both.

**IF IT DOES NOT**

See Step 12. Always start from the outside (DNS, certificate) and move inward.

**NEXT → Step 12 is your checklist when something fails. After that you repeat steps 9 and 10 for every change ("Daily routine").**

C RUN IT

# STEP 12 of 12 When something breaks: check from the outside in




**Why this step:** A connection passes through six layers. A failure at an outer layer hides everything behind it, so check them in this order and stop at the first that fails.

**NEEDS**

- Any step that failed.

→ THIS STEP →

**GIVES**

- The failing layer, and what to do about it.

## Do

Open the browser's developer tools (F12): **Console** and **Network > WS**.

| # | Layer | Quick test | If it fails |
|---|---|---|---|
| 1 | **Name** | `nslookup DOMAIN` returns VM_IP | Step 3. Wait, or try `nslookup DOMAIN 8.8.8.8`. |
| 2 | **Certificate** | Open `https://DOMAIN/` in the browser: no warning | Step 5 (use the real certificate, not `--staging`). Console text "WebSocket failed" with no detail is usually this. |
| 3 | **Firewall / network** | `sudo ufw status` shows 80 and 443. VM_IP answers `ping`. | Step 4. In NAT mode check the port forwards. |
| 4 | **Game files** | `curl -sI` shows `content-type: application/wasm` and the right `content-encoding` | Browser message "Unable to parse Build/....framework.js.gz" means the headers or double compression: use the nginx template from the repo and keep `gzip off`. |
| 5 | **Lobby** | `curl -s https://DOMAIN/api/health` | `sudo journalctl -u maze-lobby -n 30`. `room_failed_to_start`: the server binary is missing or cannot start (check name `MazeServer.x86_64`, `chmod +x`). |
| 6 | **Room approval** | The game shows a disconnect reason | `bad token`, `wrong room`, `token expired` (the token lives 60 seconds: do not wait after creating), `room full`. Also check the VM clock (Step 4). |

### More specific symptoms

| Symptom | Most likely cause |
|---|---|
| Browser error "insecure WebSocket endpoint" | The page is https but the game used `ws://`. The lobby answer must say `"secure":true`. |

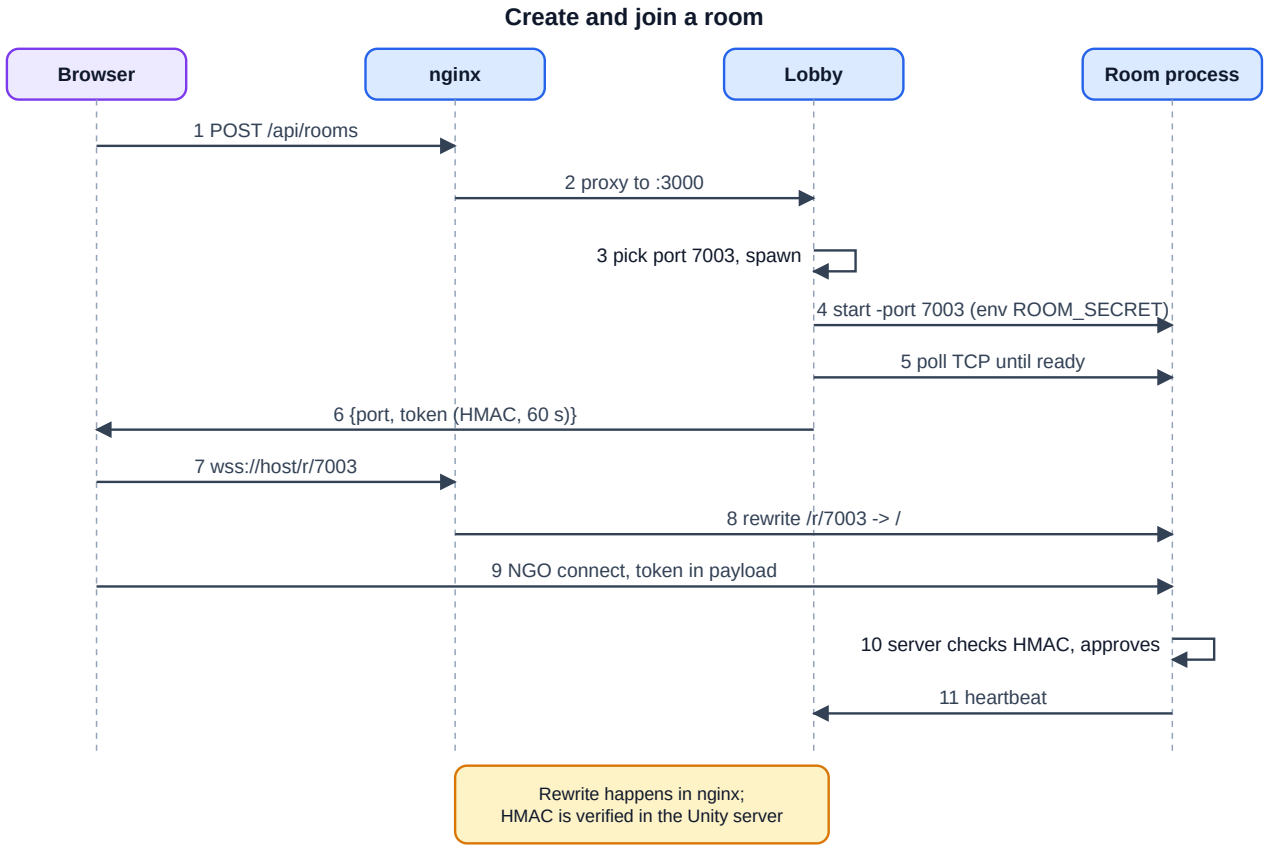
| Symptom | Most likely cause |
|---|---|
| Disconnect after about 60 seconds idle | nginx idle timeout; the template raises it to 1 hour for room paths. |
| Disconnect after minutes in a **hidden tab** | Browsers slow hidden tabs. Keep the window visible; the game should rejoin when you return. |
| Works on your PC, not on the phone | The phone resolves the private address differently or the router blocks it. Use a real certificate and the same Wi-Fi, or try another DNS. |
| Old game after publishing | Browser cache. Hard refresh (Ctrl+F5) or a private window. |

**If routing fails** (Step 11 showed `GET /` instead of `/r/7000`, or a 101 with no connection): tell me the exact `rooms.log` line. The repository already has two fallbacks, kept out of the main path to stay simple: a public port range (`server/lobby/tools/gen-port-range.mjs`) and a small Node router (`server/lobby/tools/ws-router.mjs`). They are described in the long report, chapter 5.9.

> **CHECK IT WORKED**
>
> You can name the layer that fails.

# How it works

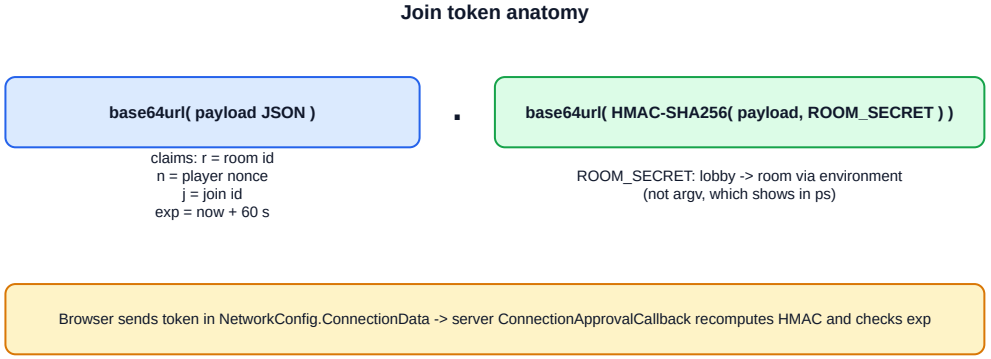
## How the pieces talk (one player creates a room)



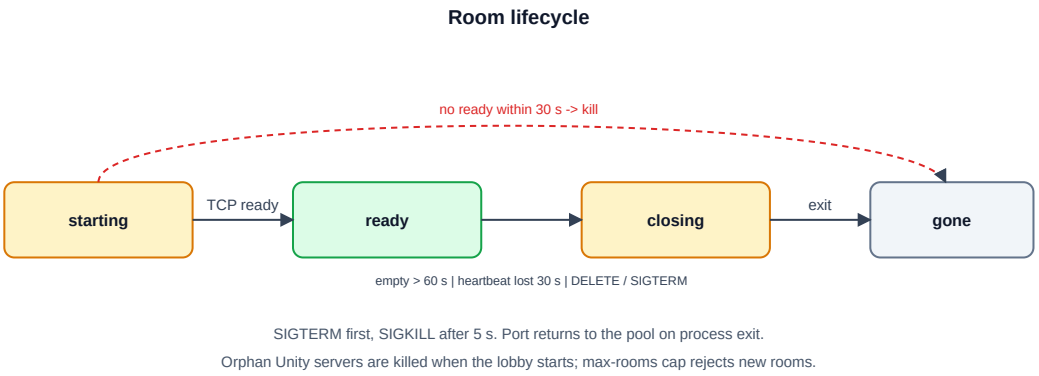

Read the picture from top to bottom. Two checks protect the room:

- **nginx rewrite (arrow 8):** the browser asks for `/r/7003`. The room program only knows `/`. nginx translates, so the browser sees one address and one port for everything.
- **Join token (arrows 6 and 10):** the lobby creates a token signed with a secret that only the lobby and the room share (given to the room as an environment variable). The room recomputes the signature. No valid token, no entry. Tokens expire after 60 seconds and work once.

## The join token

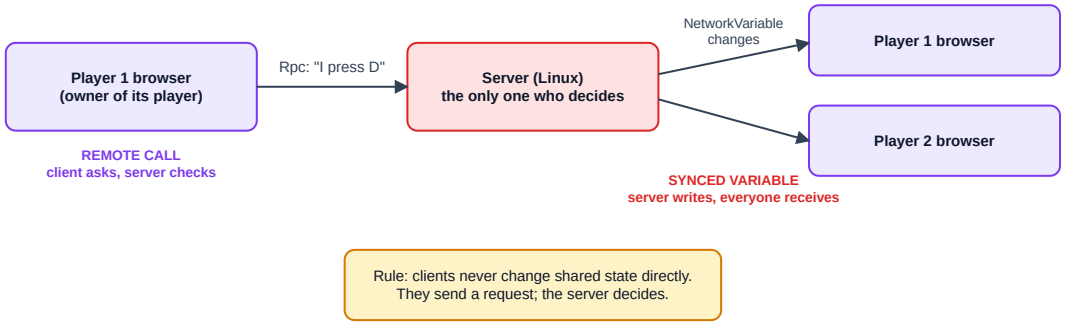


## The life of a room




The lobby is the manager of the rooms. It starts a room when someone presses Create, waits until the room accepts connections, and watches it. A room that stays empty for 60 seconds, or stops sending its 5-second heartbeat for 30 seconds, is stopped. When the lobby itself restarts it kills leftover rooms first. The lobby keeps at most `MAX_ROOMS` rooms (default 8, set in `/etc/maze/lobby.env`).

## Synced variables and remote calls (the Unity scripts)




Who is allowed to change what

There are only two tools, and they go in opposite directions:

| Tool | Direction | Use it for | Example in the repo |
|---|---|---|---|
| **NetworkVariable** | server → everyone | State everybody must see: score, colour, game phase | `PlayerState.Score`, `GameFlow.Phase` |
| **Rpc** (remote call) | client → server (or server → clients) | A request or an event | `MoverNet.SendInputRpc`, `Chat.SendChatRpc` |

The pattern that keeps the game fair:

1. The player presses D. The client does **not** move the square. It calls `SendInputRpc(direction)` (attribute `[Rpc(SendTo.Server)]`).
2. The server receives the call, limits the value (`Vector2.ClampMagnitude`), and moves the player.
3. The new position travels to everybody through the `NetworkTransform`, and variables like `Score` travel as `NetworkVariable` values.

Three rules to remember when you write your own scripts:

- Method names of remote calls **must end in `Rpc`**.
- Write a `NetworkVariable` only on the server (`if (IsServer)`). The default write permission is server-only.
- In every Rpc the server must ask "is this player allowed?" and "is this value sane?". Use `RpcParams.Receive.SenderClientId` to learn who asked. Never trust an id the client sends.

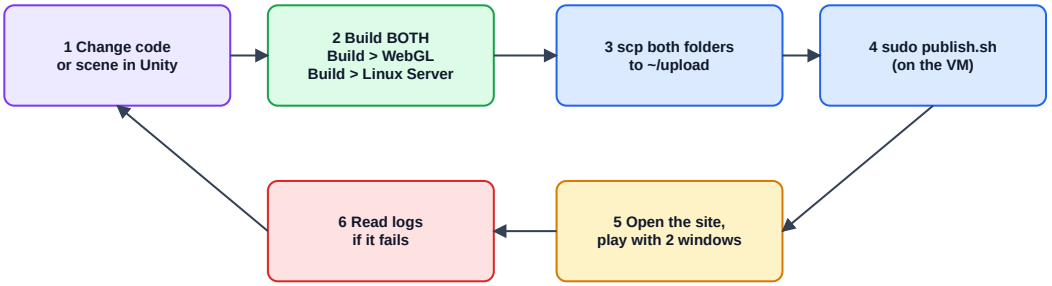
Small example from `Chat.cs`. One remote call goes to the server, the server answers all clients with a second call:

```
[Rpc(SendTo.Server)]
public void SendChatRpc(FixedString128Bytes msg, RpcParams p = default)
{
    if (msg.Length == 0) return;                          // the server checks the input
    BroadcastChatRpc(p.Receive.SenderClientId, msg);      // then tells everyone
}

[Rpc(SendTo.ClientsAndHost)]
void BroadcastChatRpc(ulong from, FixedString128Bytes msg) => Debug.Log($"[{from}] {msg}");
```

> **Important for WebGL:** a browser can only be a client. It can never "host". That is why every room is a separate Linux server program. A player who creates a room is just the first one in it; the server keeps the authority.

## Daily routine after the first setup



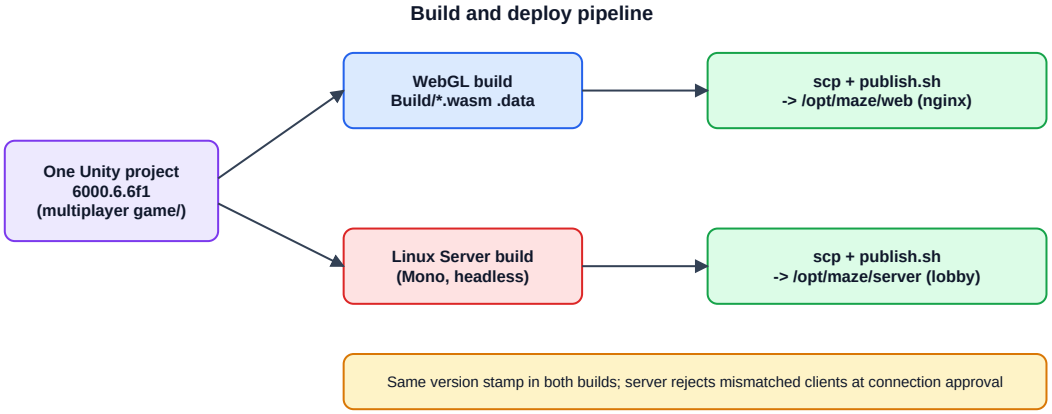

Why build both every time: the browser game and the room server must come from the same code version.

Steps 1 to 7 are done once. After that your work is this loop. It takes a few minutes.

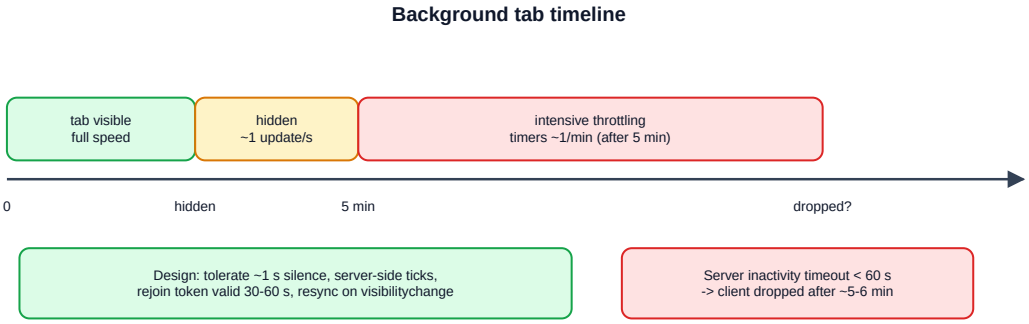
1. Change code or the scene in Unity.
2. Build both (Step 9).
3. `scp` both folders (Step 10, first block).
4. `sudo /opt/maze/bin/publish.sh` on the VM.
5. Play with two windows (Step 11). If it fails, Step 12.

Changing only the **lobby** (files in `server/lobby`)? Upload `server` again, and on the VM run `sudo bash ~/upload/deploy/install-on-vm.sh DOMAIN`. It is safe to run again: it keeps your secrets.

## Build and upload pipeline



# Time and background tabs




Why a window hidden for minutes can disconnect

# File map of the repository

| Path | What it is | Step |
|---|---|---|
| `server/lobby/` | The Node lobby: create / join / list rooms, start and stop room programs. | **6** |
| `server/lobby/tools/` | Stand-in room (fake-room.sh) and two fallback routing tools. | **6, 7, 12** |
| `deploy/install-on-vm.sh` | Run on the VM once: installs lobby, nginx site, service. | **6** |
| `deploy/publish-on-vm.sh` | Run on the VM after each upload: puts builds in place. | **10** |
| `deploy/nginx-maze.conf.template` | The nginx settings (DOMAIN is filled in by the installer). | **6** |
| `deploy/maze-lobby.service` | The systemd definition of the lobby service. | **6** |
| `deploy/lobby.env.example` | Documentation of the settings in /etc/maze/lobby.env. | **6** |
| `unity-scripts/Net/` | ServerBootstrap, ClientConnect, LobbyClient. | **8** |
| `unity-scripts/Editor/BuildScript.cs` | Menu entries Build > WebGL and Build > Linux Server. | **8, 9** |
| `unity-scripts/Gameplay/` | PlayerState, MoverNet, Chat, GameFlow: examples of synced variables and Rpcs. | **8** |
| `reports/` | Research reports with sources and the long list of what is proven. | **-** |

## If you need more detail

The long research report with sources, every code block labelled VERIFIED / TESTED / UNTESTED, security hardening, CI and the full list of pitfalls is in `reports/Unity WebGL multiplayer implementation guide.md` and the PDF next to it.